OpsPilot: Product Requirements

Agentic operations assistant for retail request handling

1. Problem and target users

Problem. A growing retail company runs operations on email, spreadsheets and personal follow-ups. Requests about delayed deliveries, stock shortages, supplier issues, urgent approvals and customer commitments arrive in shared inboxes. For each one, a person has to read it, find the facts, check stock, judge the impact, work out options, chase an approver before the deadline, update stakeholders and report upward. A typical example: *“500 display units for the Pune launch are 10 days late, 120 are in stock, approve any extra cost by 4 PM today.”*

Why it hurts. It is slow: one request takes about an hour of reading, checking and chasing. Progress depends on whoever happens to be online, deadlines are missed while approvals sit in someone’s inbox, the same issue is worked twice when several people report it, and nobody can later show who decided what, on which facts.

Goal. Cut the time from receiving a request to having a reviewed, approved plan, without giving up human control over money and commitments, and keep a complete record of every AI and human step.

Non-goals. Replacing the people who decide; running purchasing, ERP or warehouse systems; sending anything automatically.

User	Today	Needs from OpsPilot
Operations analyst	Reads every email, checks stock, drafts options, chases people	Checked facts with their source, flags for what is missing or odd, ready options, drafted emails
Manager / regional head	Gets forwarded threads with no clear ask or cost	Only the decisions within their limit, with cost, evidence, risk and a due time
Admin / operations lead	No view of workload or of who did what	Audit trail, control over failure paths, demo and pilot settings
Senior leadership	Asks for status by email	A short summary of open cases, risks and overdue approvals

Key user stories. - As an analyst, I import an email and within a minute see the facts, each with the quote it came from, so I can trust or correct them. - As an analyst, I am told what is missing or conflicting, and a clarification email is drafted for me. - As an analyst, I choose a recommended option, or override it with a reason, and the system routes it to the right approver. - As an approver, I see only approvals within my limit, with the due time, and cannot approve my own choice. - As anyone working a case, I always see the next step and who it is waiting for. - As leadership, I get a management summary in business terms on demand.

2. Proposed workflow

Step	Who	What happens	Status after
1. Receive	Person	Email opened in the inbox and imported, or a request typed in	processing
2. Extract	Intake agent	Facts (place, item, quantities, supplier, delay, deadline, ask, intents, dependencies) with an exact quote for each	processing
3. Verify	Rules (code)	Required fields, quote really in the email, inventory comparison, duplicates, injection patterns	processing
4. Assess	Risk agent + rules	Urgency, impact, reasons; rule score beside it; higher level is final; confidence computed	needs info / review

Step	Who	What happens	Status after
5. Recommend	Planner agent	Two or three options with extra cost, time to effect, rationale and risks; one recommended	review
6. Decide	Person	Accept, choose another (with reason), or write a custom action	pending approval / in progress
7. Approve	Approver	Approve, or reject with a reason, which sends the planner back for new options; overdue approvals escalate	in progress / review
8. Execute and resolve	Person	Carry out the action; resolve with a note on what was done; requester update drafted	resolved
9. Close and report	Person, reporter agent	Close; case summary and portfolio report for management	closed

At any open stage a request can be **cancelled** (pending approvals are cancelled and approvers notified), its facts can be **corrected** (analysis re-runs), or a **supplier update** can change the delay (pending approvals are cancelled and the case is reassessed). The communications agent drafts emails to the requester, supplier, store team or approver at any time; a person sends them. Every request shows its **next step** and who it is waiting for.

3. Agent design

Agents and coordination. Five agents, each with one job and a strict JSON output: *intake* (extraction), *risk* (urgency and impact), *planner* (options), *communications* (stakeholder emails) and *reporter* (case and portfolio summaries). Agents never call each other. A plain workflow runs them in a fixed order, and they share context through the stored request (facts, evidence, flags, risk, options, rejected options). Each agent receives only the slice it needs; for example, the planner gets the facts, the final risk, the verification result, matching inventory, suppliers and the options an approver already rejected. Deterministic checks sit between agents, so nothing the model says reaches a decision without being checked.

When human approval is mandatory. Agents cannot approve, send or close anything. A chosen action needs an approver when **any** of these holds: extra cost above zero, final risk high or critical, or confidence below 0.6. The approver is the lowest role whose limit covers the cost (manager Rs 50,000, regional head Rs 5,00,000, admin unlimited). The person who chose the action can never approve it. Overriding the recommendation, rejecting, cancelling and resolving all need a written reason.

How confidence is determined. It is computed in code from evidence, never taken from the model: start at 1.0 and subtract 0.1 per missing required field, 0.15 per value the email does not support, 0.15 per conflict with inventory, 0.2 when the risk agent and the rules are two or more levels apart, 0.3 for suspected prompt injection and 0.3 when the planner reports insufficient evidence. Below 0.6, no option is preselected, a reason is required and an approver must sign off.

Unsupported or incorrect output. Every reply is parsed and checked against a schema; an invalid reply gets one retry with the error, then the step fails and the request waits for a person. Each extracted value must come with a quote that appears in the email (and numbers must appear in the quote), otherwise it is removed and flagged. Risk is rated twice and the higher level wins. Planner costs are clamped to non-negative whole rupees. Users can correct any field; the correction is recorded with its reason and the analysis re-runs. Email text is passed as untrusted data and also screened by pattern rules, so an injection is caught even if the model misses it.

Monitoring and audit. An append-only audit table records every agent run (agent, model, duration, attempt, success, error, prompt version), every human action with before and after values and reasons, system actions (escalation, failed sends) and refused attempts. It is shown on each request. A health endpoint reports database and model status; per-user limits stop one user exhausting the model.

4. Requirements

Functional

#	Requirement	Acceptance check
F1	Read an email from the inbox (with preview) or a typed request	Six sample emails importable; manual form accepted
F2	Extract facts, deadlines and dependencies with evidence	Each value shows its quote; relative dates resolved in IST
F3	Assess urgency, impact and risk	Agent, rule and final level shown with reasons
F4	Identify missing, conflicting, duplicate, unsupported and unsafe content	Flags shown; clarification email drafted for missing fields
F5	Recommend options with rationale	Two or three options with cost, timing, rationale, risks
F6	Route for approval or escalation	Approver chosen by amount; overdue approval escalates to next role
F7	Review and override	Accept, choose other, custom action or edit a fact; reason required
F8	Track until closure	Statuses from processing to closed or cancelled; next step shown; resolve needs an outcome note
F9	Communications and management summaries	Drafts per audience; case summary; portfolio report
F10	Audit trail of AI and human actions	Every agent run and user action recorded and visible
F11	Roles and limits	Analyst, manager, regional head, admin; checked on the server
F12	Failure drills	Admin can advance the clock, take the model offline, make mail fail and reset demo data

Non-functional

Area	Requirement
Safety	Agents never approve, send or close; email is untrusted data; model output validated before use
Security	Salted password hashes; HttpOnly SameSite session cookie over HTTPS; server-side role checks; parameterised SQL; escaped UI output; input length limits
Auditability	Audit rows never updated or deleted; prompt version recorded with each run
Availability	If the model fails, the request stays usable: retry, or fill in facts by hand
Performance	Analysis under one minute per request; the interface stays responsive while it runs
Cost control	At most 30 analysis actions per user per hour; one model call at a time
Operability	One service and one database file; health endpoint; container image; 75 automated tests with a fake model

5. Assumptions and trade-offs

Assumptions. Requests are in English; money is in Indian rupees and time is IST; inventory, stores and suppliers come from internal systems (seeded for the prototype); approval limits follow a simple amount matrix; mail is mocked for the prototype.

Decision	Why	Trade-off
Own small pipeline instead of an agent framework	Fixed order is enough; easier to test, explain and audit	No dynamic planning or tool use by agents
Code owns grounding, final risk, confidence and limits	The model can be wrong or manipulated	More rules to maintain as the business changes
Hosted LLM (Claude Sonnet) called through its command-line client, one call at a time	No API keys on the server; fits a small host	Tens of seconds per request; queues under load
FastAPI, SQLite, plain HTML and JavaScript	Fast to build, run and review	Not for many concurrent writers; move to Postgres for production
Mock inbox and outbox with failure switch	The brief does not need real mail; failure paths stay testable	IMAP/SMTP or Gmail API needed for a pilot
Planner estimates costs	No price list in scope	Approvers must check the figure
Shared demo accounts with daily data reset	Evaluators can try every role	Evaluators can see each other’s actions

6. Edge-case handling

All fifteen listed scenarios are handled and covered by automated tests.

Scenario	Detect	Communicate	Recover	Record (audit)
Missing information	Required-field check	Flag; status needs info	Clarification email drafted; edit fields; retry	assessed, field_corrected
Conflicting information	Email quantity vs inventory	Flag with both numbers; confidence down	Correct a field or decide with the conflict shown	assessed
Duplicate request	Same place and item, or similar text	Flag naming the earlier request	Cancel the duplicate	assessed, status_changed
Unclear intent	Several intents or no clear ask	All intents listed; missing fields flagged	Edit facts; choose or write the action	extracted
Incorrect extraction	User correction	Value marked “set by user”	Analysis re-runs on corrected facts	field_corrected (before, after, reason)
Low confidence	Score below 0.6	Warning; nothing preselected	Reason required; approver must sign off	assessed, approval_requested
Approval delay	Due time passes (checked each minute)	Approval shown overdue; next approver notified	Escalates to next role with a new 2-hour deadline	approval_escalated
Approval rejection	Approver rejects with reason	Option marked rejected	Planner proposes alternatives without it	approval_rejected
Changing circumstances	Supplier update entered	Facts updated	Pending approval cancelled; case reassessed	circumstances_changed, approval_cancelled
Unavailable model	Error, timeout or invalid output twice	“AI step failed” flag	Retry, or fill facts by hand; rule-based risk used if the risk agent fails	agent run with error, processing_failed
Unauthorized action	Role limit or own choice	Refused with a clear message (403)	Another approver decides	unauthorized_attempt

Scenario	Detect	Communicate	Recover	Record (audit)
Prompt injection	Pattern rules plus model flag	Red flag; confidence reduced	Instructions ignored; person reviews	assessed
Communication failure	Mail server error on send	Message marked failed	Retry send	message_failed, message_sent
Contradictory agents	Agent and rule risk two or more levels apart	Flag with both levels	Higher level used; person reviews	assessed
Request cancellation	Cancel with reason	Status cancelled	Pending approvals cancelled; approvers notified	status_changed, approval_cancelled

7. Success metrics, MVP scope and roadmap

Targets for a pilot, not measured results:

Metric	Baseline (manual)	Pilot target	How measured
Receipt to reviewed recommendation	About 60 minutes	Under 10 minutes	Audit: created to recommendation_accepted
Approvals decided before the deadline	Unknown	Above 90%	Approval due and decided times
Extracted fields corrected by users	n/a	Under 10%	field_corrected / extracted fields
Recommendations accepted without override	n/a	Above 60%	accepted vs overridden
Requests with a complete audit trail	Near 0%	100%	Audit coverage per request
Unauthorized approvals completed	Unknown	0	Server checks; unauthorized_attempt count

MVP (built). Everything in section 4: inbox and manual intake, five agents, rules, confidence, approvals with limits and escalation, review and override, tracking to closure, communications and reports, audit trail, roles, failure drills, sample data, tests, container image and a live demo.

Next. Real mailbox and outbound mail with delivery retries; live inventory, supplier and price data; Postgres and a job queue; single sign-on and session expiry; notifications in chat; dashboards on cycle time and approvals.

Risks. Wrong or manipulated model output (mitigated by grounding, rule checks, confidence and mandatory approval); cost estimates without a price list (approver checks; price catalogue next); slow model calls under load (queueing now, API with concurrency later); adoption (start with one region and one request type, measure against the baseline above).